

1. Análisis de Complejidad Computacional

1.1. Introducción

La solución implementada para el problema *Insertion Sort Advanced Analysis* utiliza el algoritmo *Merge Sort* para contar el número de inversiones presentes en un arreglo.

Una inversión corresponde a un par de posiciones (i, j) tal que:

$$i < j \quad \text{y} \quad arr[i] > arr[j]$$

El número de inversiones coincide con el número de desplazamientos (*shifts*) que realizaría el algoritmo Insertion Sort. Por esta razón, el problema puede resolverse eficientemente mediante un proceso de división y combinación basado en Merge Sort.

1.2. Análisis complejidad computacional

La solución implementada utiliza el algoritmo *Merge Sort*, el cual divide recursivamente el arreglo en dos mitades hasta obtener subarreglos de un único elemento. Posteriormente, durante la fase de combinación (*merge*), además de ordenar los elementos, se contabilizan las inversiones presentes entre ambas mitades.

El tiempo de ejecución del algoritmo puede modelarse mediante la siguiente recurrencia:

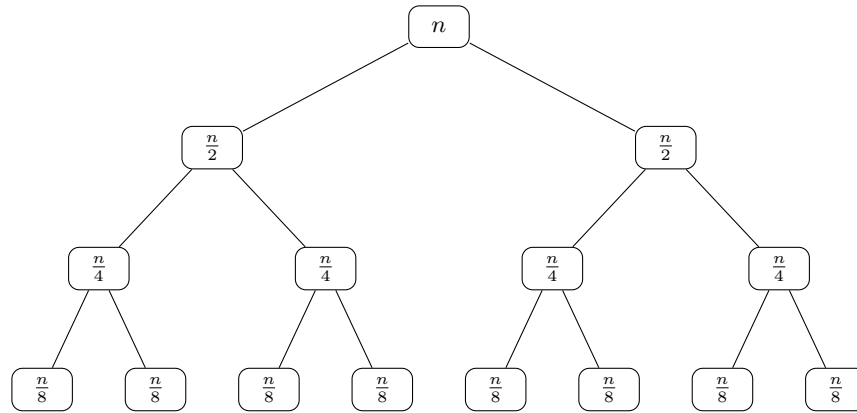


Figura 1. Árbol de recursión del algoritmo Merge Sort.

Cada nivel del árbol realiza un trabajo total de n operaciones. Como el árbol tiene una profundidad de $\log_2(n)$ niveles, el tiempo total de ejecución puede expresarse como:

$$T(n) = \underbrace{n + n + n + \cdots + n}_{\log_2(n) \text{ niveles}}$$

o, de forma equivalente,

$$T(n) = \sum_{i=1}^{\log_2(n)} n = n \sum_{i=1}^{\log_2(n)} 1 = n \log_2(n)$$

Por lo tanto,

$$\boxed{T(n) \in O(n \log n)}$$

1.3. Conclusiones

El uso de Merge Sort permite contar inversiones de forma mucho más eficiente que una solución basada en comparaciones directas entre todos los pares de elementos.

El análisis mediante sumatorias demuestra que cada nivel del árbol de recursión realiza un trabajo lineal de tamaño n , mientras que la profundidad del árbol es $\log_2 n$. Como consecuencia, el tiempo total de ejecución es:

$$O(n \log n)$$